

Component description

In this project we have 6 components. From the resources in its learning we were given the rx_mod_rtl.vhd and tx_mod_fsm.vhd. These either receive serial data and convert it to a bus, or receive a bus and send it as serial data respectively. In order for these modules to communicate properly, a module uart_loop_mod_fsm.vhd was also given in the resources. Together these modules can receive 10 bits on a serial line, including a stop and start bit (thus making the data 8 bits), process it, and pass it to a serial output. A clk_divider.vhd file was also given, which translates the clock speed of 125 MHz on the FPGA to a baud rate of 115200. From the slides, a ram module was also given. This was adjusted a bit from slides. Originally it contained 16 places of 4 bit information, which was adjusted to 8 bit information in order to store a full byte.

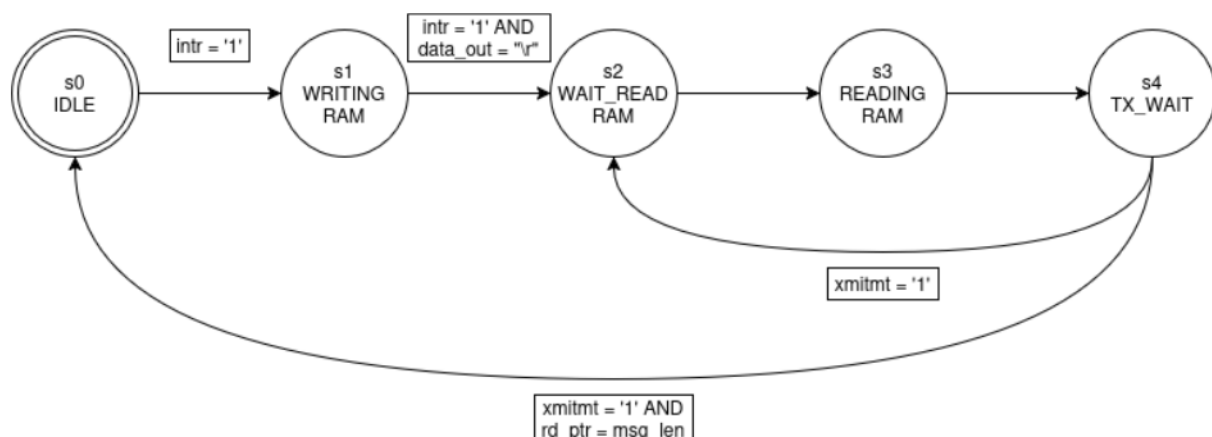
A single component was created in order to solve the assignment: the ram_ctrl.vhd, or ram controller.

Controller

The controller is designed with inspiration to the uart_loop_mod, but in addition can communicate with the ram block. This means it shares the same names for the ports of the uart_loop, but in addition has addr, din, wr, and dout in order to communicate with the ram block. The controller has 5 states:

- s0: IDLE
- s1: WRITING too ram
- s2: wait_read, short state between write and read from ram.
- s3: READING from ram, send to TX
- s4: wait for TX to be ready to receive

```
entity ram_ctrl is
  port(
    clkfast    : in  std_logic;
    rst        : in  std_logic;
    -- from RX
    intr       : in  std_logic;
    data_out   : in  std_logic_vector(7 downto 0);
    -- from TX
    xmitmt     : in  std_logic;
    -- to TX
    data_in    : out std_logic_vector(7 downto 0);
    shift_load : out std_logic;
    -- to/from RAM
    addr       : out std_logic_vector(3 downto 0);
    din        : out std_logic_vector(7 downto 0);
    wr         : out std_logic;
    dout       : in  std_logic_vector(7 downto 0)
  );
end ram_ctrl;
```

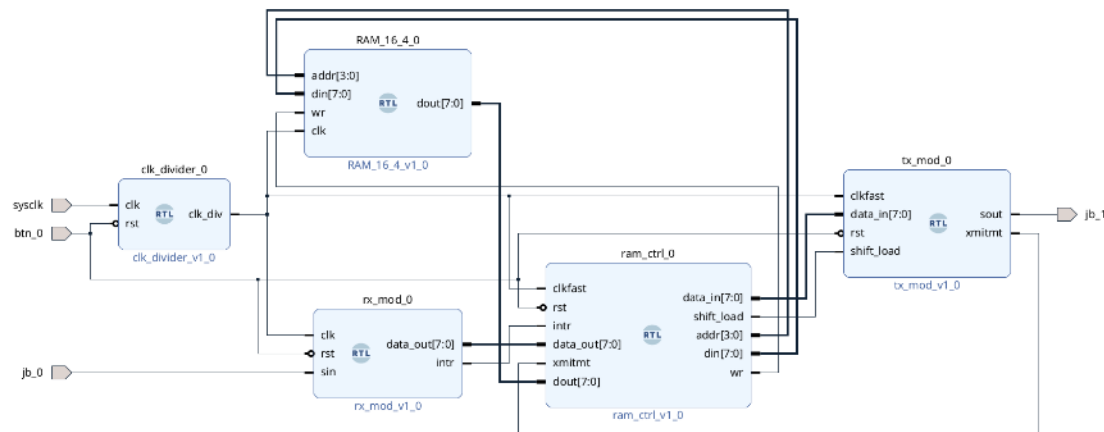


At the start, the ram controller is in the idle state. If it gets an interrupt from the RX module, the nextstate is set to s1. in this state, the write pin is pulled high, telling the ram to be ready to receive. The address in which the information is supposed to be stored is passed to the ram block alongside the data, which comes from the RX module. When in s1, the controller checks if the input from the RX module is the carriage return. If this happens, the state is set to s2, indicating all information has been received and stored. When in s1, each time a byte is stored, the internal signal wr_ptr is incremented, thereby storing the data in sequence inside the ram.

When in s2, the controller informs the ram that its ready to read data by pulling the wr pin low. In addition, it sets the addr to the internal signal rd_ptr, which at the beginning is 0. Then the controller goes to s3, saving input from the ram to an internal signal tx_buf and increments the rd_ptr. It holds the addr line to RAM as rd_ptr, meanwhile sending the data to TX and '1' on the shift_load pin. When the rd_ptr is set, the nextstate process runs and sets the state to s4.

In s4, the module waits for the xmitmt to be pulled high. This tells the system that the TX module is done sending and is ready to receive again. Then the state goes back to s2. If the rd_ptr is equal to msg_len, then all the information in the ram has been transmitted and the state goes back to s0.

Block diagram & data flow



In the UART with RAM system, the UART loop module is replaced with a 16x8 RAM module, and the RAM FSM controller, which is used to write to or read from RAM at the right time. The RAM FSM controller interfaces with both RX and TX, and also the RAM module. Thus the write data flow is

- Computer → serial data → RX → 1 byte → RAM controller → RAM.

Once RX has received a carriage return, the read data flow is

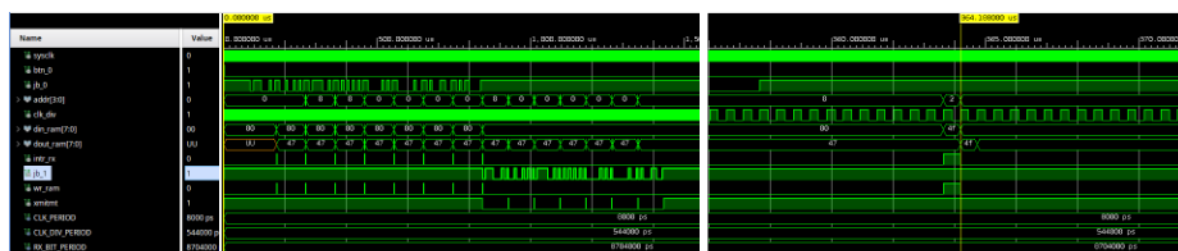
- RAM → 1 byte → RAM controller → TX → serial data.

This is passed onto the second FPGA, which uses normal UART to pass the data onto the second computer

- Serial data → RX → 1 byte → UART loop module → 1 byte → TX → serial data → computer.

Simulation

Behavioral Simulation



The behavioral simulation was done, to confirm the logic of the system. It can be seen, that the RX module receives a message "GROUP 1" in hexadecimal serial, and not long after that, we can see signals on din_ram and dout_ram, showing, that the data gets written to RAM. If we zoom in, we can see the data between the "00" signals on din_ram and the "47" signals on dout_ram. Once, RX has

received the last character, “\r”, TX starts transmitting the message, loaded from RAM. We can see the loading from RAM in action, by looking at the signals on dout_ram. One small thing to notice, is that the signal on TX, does not exactly match the signal on RX. This is evidently due to an error in the stop bit, supposed to be sent on TX, as the rest of the bits seems to lign up somewhat nicely. It can also be seen that xmitmt fires once for every byte transmitted.

Post Synthesis Simulation



On the post synthesis timing simulation, we can see almost the same behavior as the behavioral simulation. The only noticeable difference, is that xmitmt now fires for every bit sent instead of every byte. This should in theory affect the system, by TX wrongfully transmitting that it is done, and ready for another message, but we can see the signal from TX has not changed from the behavioral simulation.

Post Implementation Simulation



The post implementation timing simulation mirrors the post synthesis timing simulation exactly, showing no change in any of the signals.

Youtube demonstration of working system:

by <https://youtube.com/shorts/OAi7gT7zVAQ>

Appendix A - Constraints

```
## Clock signal 125 MHz

set_property -dict {PACKAGE_PIN H16 IOSTANDARD LVCMOS33} [get_ports sysclk]
create_clock -add -name sys_clk_pin -period 8.00 -waveform {0 4} [get_ports {
sysclk }];

##Buttons

set_property -dict { PACKAGE_PIN D19      IOSTANDARD LVCMOS33 } [get_ports { btn_0
}]; #IO_L4N_T0_35 Sch=btn[1]

##PmodB

set_property -dict { PACKAGE_PIN W14      IOSTANDARD LVCMOS33 } [get_ports { jb_0
}]; #IO_L8P_T1_34 Sch=jb_p[1]
set_property -dict { PACKAGE_PIN Y14      IOSTANDARD LVCMOS33 } [get_ports { jb_1
}];
```

Appendix B - Code

RAM Controller

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;

entity ram_ctrl is
    port(
        clkfast      : in  std_logic;
        rst           : in  std_logic;
        -- from RX
        intr          : in  std_logic;
        data_out      : in  std_logic_vector(7 downto 0);
        -- from TX
        xmitmt        : in  std_logic;
        -- to TX
        data_in       : out std_logic_vector(7 downto 0);
        shift_load    : out std_logic;
        -- to/from RAM
        addr          : out std_logic_vector(3 downto 0);
        din           : out std_logic_vector(7 downto 0);
        wr            : out std_logic;
        dout          : in  std_logic_vector(7 downto 0)
    );
```

```
end ram_ctrl;

architecture fsm of ram_ctrl is

    -- port names now match the other modules (data_out from RX, data_in to TX)

    constant TERMINATOR : std_logic_vector(7 downto 0) := x"0D"; -- carriage
return

    type STATE_TYPE is (s0, s1, s2, s3, s4);

    -- s0 = IDLE
    -- s1 = WRITING
    -- s2 = wait/read
    -- s3 = READING
    -- s4 = tx wait (wait for tx to be ready to receive)

    attribute state_vector : string;
    attribute state_vector of fsm : architecture is "current_state";

    signal current_state : STATE_TYPE;
    signal next_state    : STATE_TYPE;

    signal wr_ptr  : unsigned(3 downto 0);
    signal rd_ptr  : unsigned(3 downto 0);
    signal msg_len : unsigned(3 downto 0);
    signal tx_buf  : std_logic_vector(7 downto 0);

begin

    -----
    clocked : process(clkfast, rst)
    -----
    begin
        if (rst = '1') then
            current_state <= s0;
            wr_ptr        <= (others => '0');
            rd_ptr        <= (others => '0');
            msg_len       <= (others => '0');
            tx_buf        <= (others => '0');

        elsif (rising_edge(clkfast)) then
            current_state <= next_state;

            case current_state is

                when s0 => -- IDLE
                    wr_ptr <= (others => '0');
                    rd_ptr <= (others => '0');
                    if (intr = '1') then
```

```

        wr_ptr <= "0001"; -- first byte goes to addr 0, next
write at 1
        end if;

    when s1 => -- WRITING
        if (intr = '1') then
            if (data_out = TERMINATOR) then
                msg_len <= wr_ptr;
                rd_ptr <= (others => '0');
            else
                wr_ptr <= wr_ptr + 1;
            end if;
        end if;

    when s2 => -- WAIT_READ (RAM latency)
        null;

    when s3 => -- READING
        tx_buf <= dout;
        rd_ptr <= rd_ptr + 1;

    when s4 => -- TXWAIT
        null;

    when others =>
        null;

    end case;
end if;
end process clocked;

-----
nextstate : process(current_state, intr, data_out, xmitmt, rd_ptr, msg_len)
-----
begin
    case current_state is

        when s0 =>
            if (intr = '1') then
                next_state <= s1;
            else
                next_state <= s0;
            end if;

        when s1 =>
            if (intr = '1' and data_out = TERMINATOR) then
                next_state <= s2;
            else
                next_state <= s1;
            end if;
    end case;
end process;

```

```
when s2 =>
    next_state <= s3;

when s3 =>
    next_state <= s4;

when s4 =>
    if (xmitmt = '1') then
        if (rd_ptr = msg_len) then
            next_state <= s0;
        else
            next_state <= s2;
        end if;
    else
        next_state <= s4;
    end if;

when others =>
    next_state <= s0;

end case;
end process nextstate;

-----
output : process(current_state, wr_ptr, rd_ptr, data_out, tx_buf, intr,
dout)
-----
begin
    -- defaults
    wr      <= '0';
    shift_load <= '0';
    addr     <= (others => '0');
    din      <= (others => '0');
    data_in  <= (others => '0');

    case current_state is

        when s0 => -- IDLE - but catch first byte
            if (intr = '1') then
                wr      <= '1';
                addr <= std_logic_vector(wr_ptr);
                din    <= data_out;
            end if;

        when s1 => -- WRITING
            if (intr = '1') then
                wr      <= '1';
                addr <= std_logic_vector(wr_ptr);
                din    <= data_out;
            end if;
```

```
when s2 => -- WAIT_READ
    addr <= std_logic_vector(rd_ptr);
    wr    <= '0';

when s3 => -- READING
    addr      <= std_logic_vector(rd_ptr);
    data_in   <= dout;
    shift_load <= '1';

when s4 => -- TXWAIT
    data_in <= tx_buf;

when others =>
    null;

end case;
end process output;

end fsm;
```

RAM 16x8

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_arith.all;

entity RAM_16_4 is
    port(
        addr : in  std_logic_vector(3 downto 0);
        din   : in  std_logic_vector(7 downto 0);
        wr    : in  std_logic;
        clk   : in  std_logic;
        dout  : out std_logic_vector(7 downto 0)
    );
end RAM_16_4;

architecture Behavioral of RAM_16_4 is

    type mem_type is array (0 to 15) of std_logic_vector(7 downto 0);
    signal mem : mem_type;

    signal addr_reg : std_logic_vector(3 downto 0);

begin

    process(clk)
    begin
        if rising_edge(clk) then
```

```
    if (wr = '1') then
        mem(conv_integer(unsigned(addr))) <= din;
    end if;

    addr_reg <= addr;
end if;
end process;

dout <= mem(conv_integer(unsigned(addr_reg)));
end Behavioral;
```